

Frequent Itemset Mining: Framework Design and Performance Analysis of Apriori, PCY, and SON Algorithms

Algorithms for Massive Data – End-Of-Course Project

Leila Cielok

Data Science for Economics (Master's Degree)
II Year

June 29, 2026

I declare that this material, which I now submit for assessment, is entirely my own work and has not been taken from the work of others, save and to the extent that such work has been cited and acknowledged within the text of my work, and including any code produced using generative AI systems. I understand that plagiarism, collusion, and copying are grave and serious offences in the university and accept the penalties that would be imposed should I engage in plagiarism, collusion or copying. This assignment, or any part of it, has not been previously submitted by me or any other person for assessment on this or any other course of study.

1 Introduction

Frequent Itemset Mining (FIM) and Association Rule Mining (ARM) represent fundamental tasks in data mining and exploratory data analysis. Originally formalized for market-basket analysis to discover relationships between products purchased together by customers, these techniques have expanded significantly into diverse domains (related concepts, plagiarism, biomarkers), allowing this project to apply them to cinematic collaboration networks.

From a computational perspective, finding frequent itemsets presents a major challenge due to the combinatorial explosion of candidate generation. If a dataset contains d unique items, the total search space of potential itemsets spans an unmanageable 2^d combinations. For this reason, standard counting approaches risk to cause bottleneck due to CPU or memory limitations.

To address these scalability issues, various algorithmic paradigms have been developed, which aim at avoiding counting too many triples or larger sets when unnecessary. This project evaluates and contrasts three algorithms for finding frequent itemsets within a unified Python framework, starting from the A-Priori methodology, which systematically prunes the search space by leveraging the monotonicity principle (an itemset can only be frequent if all of its subsets are also frequent). A variant to the basic A-Priori is presented through the implementation of the PCY (Park-Chen-Yu) algorithm, which optimizes the A-Priori by applying a localized hash function to the candidate pairs and a memory-efficient bit-vector during the first pass to reduce the candidate pair search space in the second pass. Finally, the SON (Savasere, Omiecinski, and Navathe) methodology is presented: a partitioning-based framework designed for large or distributed datasets. It partitions the dataset into independent chunks to find local frequent itemsets, which are subsequently validated in a single global second pass.

By evaluating these implementations on a structured dataset under varying support thresholds and hyperparameter settings, this report characterizes the empirical performance boundaries and efficiency profiles of each approach.

2 Dataset and Pre-Processing

The empirical analysis is conducted on the IMDb Movies Dataset, which comprises the 1,000 top-rated movies and television series compiled from the official IMDb platform.¹ Each record (movie) represents a basket, and the individual items within each basket correspond to the cast members ("Star1", "Star2", "Star3", "Star4") credited in that film.

To eliminate heavy string comparison workloads during processing phases, records are translated into integers using an indexing schema:

1. Translation table (**names2int**): a lookup table mapping string actor identifiers to unique integer indexes.
2. Frequent singletons array (**frequent_items_table**): an index translation vector constructed after filtering size-1 itemsets. If an item i is determined to be frequent, it receives a new identifier starting from 1; if it fails the threshold, it is explicitly mapped to 0.
3. Reverse translation table (**newint2name**): a dynamic map used exclusively at the conclusion of execution loops to re-translate processed numerical itemset pairs or triples back to readable text strings.

¹The dataset is publicly available on Kaggle: <https://www.kaggle.com/datasets/harshitshankhdhar/imdb-dataset-of-top-1000-movies-and-tv-shows>

3 Algorithms and Implementations

3.1 Apriori Implementation

The standard `apriori` implementation proceeds level-by-level. Pairs are formed directly by computing combinations over filtered item vectors. To transition from pairs to triples, the `find_frequent_triples` subroutine evaluates combinations of size 3. It utilizes an optimization trick by casting the validated pair keys into a flat Python set, enabling constant-time $O(1)$ lookups to guarantee that a trio $\{A, B, C\}$ is only tracked if all sub-pairs $(\{A, B\}, \{B, C\}, \{A, C\})$ exist in the set.

3.2 PCY (Park-Chen-Yu) Implementation

The `pcy` variant optimizes the second pass by managing an array of buckets (`bucket_counts`) of size n_{buckets} , which was tested through an hyperparameter experiment in order to choose a reasonable number of bucket structures. During the first pass, as single items are counted, all pairs occurring in a basket are evaluated via a hash model:

$$\text{hash_value} = (\text{sorted}(\text{item1}) \cdot 31) + \text{sorted}(\text{item2}) \quad (1)$$

$$\text{bucket} = \text{hash_value} \pmod{n_{\text{buckets}}} \quad (2)$$

After that, a boolean bitmap is derived by checking whether each bucket value meets the required support. In pass 2, candidate pairs are evaluated only if their assigned hash destination maps to a `True` bit state (which indicates that their allocated bucket was found to be frequent): this allows the algorithm to cut down on the size of the set of candidate pairs to find frequent triples.

3.3 SON Implementation

The third algorithm implementation avoids both false negatives and false positives, at the cost of making two full passes over the dataset. The SON algorithm relies on a simple mathematical concept, that is if an itemset is frequent across the entire dataset, it must be frequent in at least one chunk (subset) of that dataset. Accordingly, by splitting the data into chunks, running a local A-Priori on each, and combining the results, SON guarantees to find all possible true frequent itemsets in exactly two passes, eliminating memory crashes. To account for the framework’s sampling architecture, the local support threshold is dynamically scaled down based on the chunk size: each partition representing a fraction $p \approx 0.333$ of the complete dataset ($\text{len}(\text{chunk}) = 333$) is evaluated using a localized threshold of 1.665, which is dynamically adjusted to a strict integer ceiling of 2.

$$\text{local_support} = \left\lceil \frac{\text{support_count} \cdot \text{len}(\text{chunk})}{\text{len}(\text{baskets})} \right\rceil \quad (3)$$

Pass 1 executes the local `apriori` pipeline on each partitioning, consolidating discovered local frequent patterns into a global candidate structure using union updates (`.update()`).

Because local thresholds are scaled down, some items might pass locally just by random chance (false positives). Pass 2 performs a global streaming count over the full dataset to find the true global counts for the candidates.

4 Workload Complexity Scaling via Support Manipulation

Because the dataset size is fixed at 1,000 rows by external constraints, scalability testing is conducted by manipulating the workload complexity. This is achieved by systematically reducing the support ratio across four test tiers:

$$R = \{0.05, 0.01, 0.005, 0.002\} \quad (4)$$

Lowering the support ratio broadens the search criteria, resulting in an exponential increase in candidate items, pairs, and triples. This tests how each algorithm copes under sudden, combinatorial stress.

4.1 Hyperparameter Optimization Framework

To observe execution dependencies on internal constants, hyperparameter adjustments are applied across standalone loops:

- PCY Hashing Resolution: Evaluated over $n_{\text{buckets}} \in \{20, 100, 500, 1000, 5000\}$ to observe collision penalties.
- SON Chunking Granularity: Evaluated over $m \in \{3, 6, 10, 20\}$ chunks to examine the trade-offs of localized parallel scaling.

To minimize environmental CPU noise in the execution space, each pipeline iteration is re-run across multiple identical cycles (N_RUNS), with performance metrics derived from calculated temporal means.

5 Experimental Results

5.1 Scalability Metrics Table

The empirical properties observed during execution are detailed in the compiled performance registry below.

Table 1: Performance and Itemset Generation Metrics Across Scaling Workloads

Ratio	Count	Apriori (s)	PCY (s)	SON (s)	Singletons	Pairs	Triples	Total
0.050	50	0.0043	0.0054	0.0052	0	0	0	0
0.010	10	0.0039	0.0047	0.0097	9	0	0	9
0.005	5	0.0030	0.0037	0.0436	79	3	1	83
0.002	2	0.0062	0.0089	1.5598	641	121	25	787

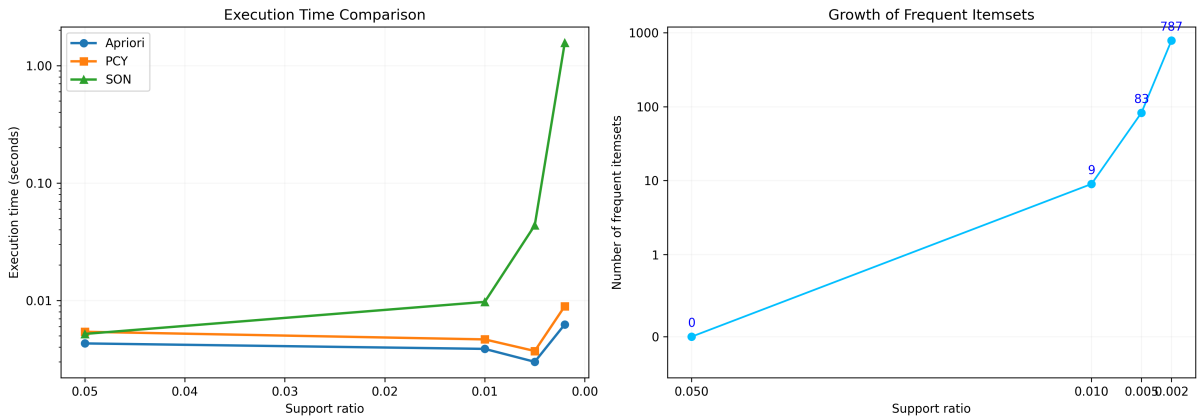


Figure 1: Empirical evaluation of framework scaling: execution runtime trends across Apriori, PCY, and SON pipelines (left) and the corresponding exponential growth in total discovered frequent itemsets (right) as the support threshold relaxes.

The empirical results compiled in Table 1 and visualized in Figure 1 illustrate a classic computational bottleneck characteristic of frequent itemset mining: the combinatorial explosion of the candidate search space as the constraint parameter relaxes.

When operating at a high support ratio ($R = 0.050$), the threshold is too restrictive for this specific 1,000-row IMDb dataset slice, yielding zero frequent items across all cardinalities. As the support ratio drops to 0.010, a modest baseline of 9 frequent singletons emerges, yet no pairs or triples manage to cross the absolute support threshold of 10 instances. At these high-to-moderate thresholds, all three algorithms perform optimally with negligible execution overhead, completing their execution in less than 0.01 seconds.

A sharp structural inversion is observed when transitioning from $R = 0.005$ to $R = 0.002$. At $R = 0.005$ (absolute count of 5), which was chosen as the baseline support threshold used for the analysis, the dataset contains 79 frequent singletons, which combine into a highly restricted set of 3 frequent pairs and 1 frequent triple. However, lowering the ratio slightly further to 0.002 (absolute count of 2) triggers an explosive proliferation of frequent structures: singletons increase by a factor of 8 ($\rightarrow 641$), frequent pairs skyrocket by a factor of 40 ($\rightarrow 121$), and frequent triples scale to 25, culminating in a dense web of 787 total frequent itemsets.

This combinatorial explosion has vastly different impacts on the performance profiles of the three implementations. Apriori and PCY exhibit exceptional resilience across the entire scaling spectrum; at $R = 0.002$, Apriori completes the mining process in just 0.0062 seconds, while PCY follows closely at 0.0089 seconds. Because the full dataset fits easily within the local memory cache (RAM), the direct level-by-level candidate validation remains highly efficient, and the hashing/bitmap management of PCY incurs a negligible tracking overhead. The SON Framework, conversely, suffers a severe performance penalty at this lowest threshold. While its execution time remains highly competitive down to $R = 0.005$ (0.0436 seconds), it surges exponentially to 1.5598 seconds at $R = 0.002$, rendering the partition-based algorithm roughly 250 times slower than the baseline Apriori.

This behavioral divergence is a direct consequence of the chunking mechanics under extreme scarcity. When partitioning 1,000 rows into multiple chunks, a global support count of 2 translates into an exceptionally low localized support threshold (often a single occurrence per chunk). Consequently, almost every localized item combination is treated as locally frequent during Pass 1. This generates an overwhelming volume of false-positive candidate pairs and triples that must be compiled globally and sequentially verified across the entire dataset during Pass 2, introducing severe processing overhead. This highlights an important architectural insight: while the divide-and-conquer strategy of the SON framework is indispensable for out-of-core distributed processing on massive disk-bound datasets, applying it to small, in-memory datasets under ultra-low support thresholds introduces a "candidate congestion" that severely degrades performance compared to standard monolithic approaches.

5.2 Hyperparameter Inferences

To isolate the operational dependencies of the PCY and SON optimization frameworks, parameter tuning experiments were conducted by systematically varying internal structural constants. Figure 2 illustrates the execution response curves under these isolated hyperparameter permutations.

The empirical behavior illustrated in the parameter curves reveals distinct structural insights into the optimization limits of both frameworks.

On the one hand, the execution profile of the PCY algorithm (left) exhibits micro-fluctuations restricted to an exceptionally narrow margin of less than half a millisecond (0.0036 to 0.0040 seconds). Because the 1,000-row IMDb dataset is compact, the hash table layout reaches its maximum pruning efficiency at 100 buckets. Scaling the parameter further does not yield additional filtering benefits; instead, it introduces minor memory allocation overhead and makes

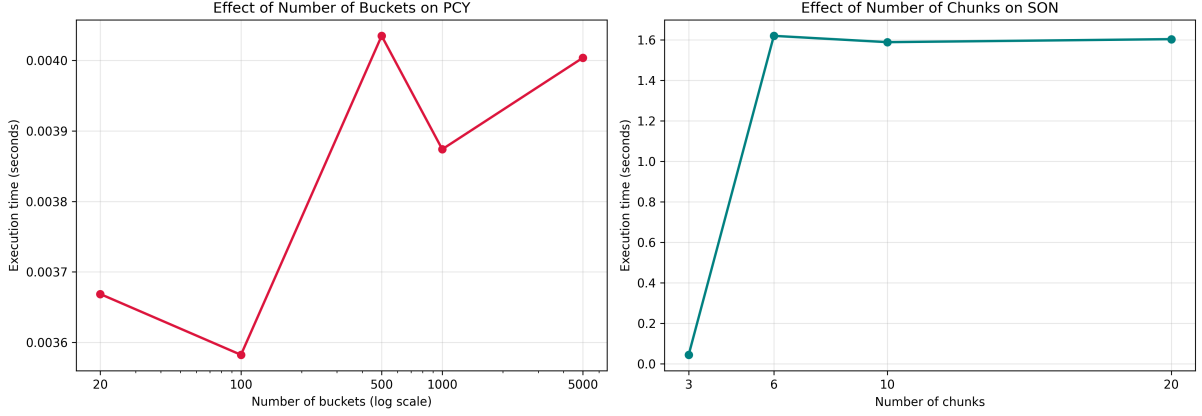


Figure 2: Empirical evaluation of hyperparameter scaling profiles: PCY runtime variation across hash bucket counts (left) and SON framework behavior under distinct data partition granularities (right).

the runtime highly sensitive to transient CPU background noise and hardware caching state, leading to the counter-intuitive performance penalty observed at 500 and 5000 buckets.

On the other hand, the partition tuning graph for the SON algorithm (right) uncovers a severe, binary threshold driven by candidate congestion. At $m = 3$ chunks, the algorithm performs efficiently (≈ 0.0626 seconds) as the local support constraints remain high enough to suppress false positives. However, as the partitioning increases to 6 or more chunks, the localized support requirement drops to minimal absolute values, allowing an overwhelming volume of candidate itemsets to pass Pass 1 simultaneously. This massively burdens the global synchronization and validation phases, causing execution times to skyrocket and plateau heavily around 1.46–1.62 seconds.